P1861R1
Secure Networking in C++

Published Proposal, 2020-05-11

This version:
http://wg21.link/P1861R1

Authors:
Alex Christensen (Apple)
JF Bastien (Apple)
Scott Herscher (Apple)

Audience:
SG4, LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:
https://github.com/achristensen07/papers/blob/master/source/p1861r1.bs

---

Table of Contents

§ 1. Abstract

A description of how a C++ networking library can elegantly support Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS) by default, as well as allow future expansion to include protocols such as QUIC.

§ 2. Introduction

An interface to modern computer networks that is intended to maintain API and ABI stability must be designed for modern protocols. Indeed, networking protocols have evolved substantially since the

Berkeley Socket APIs were originally created. Of utmost importance is support for security by default, as outlined in [p1860R0].

These design principles were discussed with the Committee's networking study group, which asked the authors to what extent the IETF TAPS framework could be used as a basis, and what the resulting API would look like. This document presents an embodiment of TAPS into a modern C++ paradigm. We believe it also matches well with current industry practice (such as Apple's Network.framework API). As a proof of feasibility, a working implementation of this interface was written as a wrapper around Network.framework. Other implementations are possible, as discussed in § 8 Implementability. The suggested API poses very little burden on C++ Standard Library maintainers.

Notable differences between this approach and the current networking proposal in [N4771] include:

1. zero-configuration networking endpoints in the `dnssd` class.
2. The `path_monitor` class, which allows programs to receive notifications of changes in WiFi connectivity, or use of cellular data.
3. `options` and `metadata` objects that allow configuration and reading of protocol-specific properties.
4. A clean interface to add security by default as described in [p1860R0].

These differences stem from following the IETF TAPS design.

Additionally, this suggested API has integration with coroutines, which allows elegant asynchronous programming. Coroutines were added to C++20, it is therefore sensible to use them.

The § 5 Suggested API and be used to create a variety of applications. We offer a few examples:

- § 6.1 Client
- § 6.2 Server
- § 6.3 Path Listener
- § 6.4 Protocol Framer

§ 3. Version History

- [P1861R0]: Minimal diff from [N4771] to support TLS and DTLS in a way that is not great.
- P1861R1: Initial suggested API based on IETF TAPS.

§ 4. Disclaimer

This paper uses `std::is_invocable_r_v` instead of concepts, and it uses `std::string_view` and `std::vector` in some places instead of ranges. The author was working with an incomplete implementation of C++20 at the time, and these could easily be updated in a future revision. It also doesn't have all the `noexcept` annotations that it could. Please try to look past that at the shape of an interface to the network.

§ 5. Suggested API

§ 5.1. `net::awaitable`

This object is a simple template that is returned when there is a possibility of asynchrony, which allows the caller to either `co_await` the result when using coroutines or call `.then` on the result when not using coroutines. This could probably use another awaitable type in the STL, and this is not directly related to networking, but networking does need a medium of asynchrony such as this.

```
namespace net {

template<typename T>
class awaitable {
public:
  awaitable();
  awaitable(const awaitable&);
  awaitable(awaitable&&);
  awaitable& operator=(const awaitable&);
  awaitable& operator=(awaitable&&);

  bool await_ready() const noexcept;
  void await_suspend(std::experimental::coroutine_handle<>);
  T&& await_resume();

  template<typename Handler, typename =
    std::enable_if_t<std::is_invocable_r_v<void, Handler, T>>>
  void then(Handler&&);
};

}
```

§ 5.2. `net::buffer`

This object represents 0 or more sections of contiguous memory used for sending and receiving data.

```
namespace net {

class buffer {
public:
  buffer();
  buffer(const void*, size_t);
  buffer(const char*);
  buffer(std::string_view);
  buffer(const buffer&);
  buffer(buffer&&);
  buffer& operator=(const buffer&);
  buffer& operator=(buffer&&);

  template<typename Handler, typename =
    std::enable_if_t<std::is_invocable_r_v<void, Handler, const uint8_t
  void get(Handler&& handler) const;
};

}
```

§ 5.3. `net::connection`

This object represents a connection with an external client or server. This connection can be used to send or receive messages. Note that UDP also uses connections in this model even though UDP is a "connectionless" protocol. This object still sends and receives UDP packets even though no handshake is necessary.

The `receive` method receives data even if the data is incomplete. This is much more common on the internet, where streams of data that never complete are common. The `receive_complete` method will only resume the awaitable when the connected peer indicates the complete method has been sent.

```
namespace net {

class connection {
public:
  enum class state {
    setup,
    waiting,
    preparing,
    ready,
    failed,
    cancelled
  };

  connection(endpoint, parameters, workqueue);

  template<typename Handler, typename =
    std::enable_if_t<std::is_invocable_r_v<void, Handler, state, std::e
  connection& on_state_changed(Handler&&);

  template<typename Handler, typename =
    std::enable_if_t<std::is_invocable_r_v<void, Handler, bool>>>
  connection& on_viability_changed(Handler&&);

  template<typename Handler, typename =
    std::enable_if_t<std::is_invocable_r_v<void, Handler, bool>>>
  connection& on_better_path_changed(Handler&&);

  connection& start();

  awaitable<expected<void, std::error_code>>
    send(message);
  awaitable<expected<void, std::error_code>>
    send(const std::vector<protocol::metadata>&, bool is_complete = tru
  awaitable<expected<void, std::error_code>>
    send(buffer, const std::vector<protocol::metadata>&, bool is_comple
  awaitable<expected<void, std::error_code>>
    send(buffer, message::context, bool is_complete = true)

  awaitable<expected<message, std::error_code>>
    receive_complete();
  awaitable<expected<message, std::error_code>>
    receive(std::size_t min_incomplete_length = 1,
```

```
        std::size_t max_length = std::numeric_limits<std::size_t>::max())
    void cancel();
};

}
```

§ 5.4. `net::endpoint`

This object represents an endpoint to be connected with. There are currently three types: host (such as `www.apple.com`), address (such as `127.0.0.1`) and dnssd (such as a Bonjour service). All three can be used to initiate a connection.

```
namespace net {

class endpoint {
public:
  class host;
  class address;
  class dnssd;
};

class endpoint::host : public endpoint {
public:
  host(std::string_view name, std::uint16_t port);
};

class endpoint::address : public endpoint {
public:
  address(const sockaddr&);
};

class endpoint::dnssd : public endpoint {
public:
  dnssd(std::string_view name, std::string_view type, std::string_view (
};

}
```

§ 5.5. `net::expected`

This object contains [p0323r7] in the `net` namespace. It is perfect for networking, when sending can either result in nothing or an error, and receiving can either result in a message or an error. It should be standardized and its references in the connection object should be changed to use `std::expected` instead of `net::expected`.

§ 5.6. `net::interface`

This object represents a network interface, such as WiFi or ethernet. Modern devices often have more than one such interface, and it is useful to be able to specify which one to use or to listen for events indicating when interface viability changes, such as when the WiFi is turned off but ethernet is still plugged in.

```
namespace net {

class interface {
public:
  enum class type {
    other,
    wifi,
    cellular,
    wired_ethernet,
    loopback
  };

  interface(const interface&);
  interface(interface&&);
  interface& operator=(const interface&);
  interface& operator=(interface&&);

  type type() const;
  std::string name() const;
  std::size_t index() const;
};

}
```

§ 5.7. `net::listener`

This object that can be used to listen for incoming connections. The constructor without a port receives a port assigned from the system, which can then be queried with the port getter.

```
namespace net {

class listener {
public:
  enum class state {
    setup,
    waiting,
    ready,
    failed,
    cancelled
  };

  listener(parameters, workqueue);
  listener(std::uint16_t port, parameters, workqueue);
  listener(const listener&);
  listener(listener&&);
  listener& operator=(const listener&);
  listener& operator=(listener&&);

  template<typename Handler, typename =
    std::enable_if_t<std::is_invocable_r_v<void, Handler, connection>>>
  listener& on_new_connection(Handler&&);

  template<typename Handler, typename =
    std::enable_if_t<std::is_invocable_r_v<void, Handler, state, std::e
  listener& on_state_changed(Handler&&);

  void start();
  std::uint16_t port() const;
};

}
```

§ 5.8. `net::message::context`

This object represents a set of protocol metadata associated with the act of sending or receiving data on a connection.

```
namespace net {

class message::context {
public:

    static context final_message();
    static context default_stream();
    static context default_message();

    context();

    template<typename Protocol> typename Protocol::metadata metadata();

    template<typename Protocol> const typename Protocol::metadata metad

    void set_metadata(const std::vector<protocol::metadata>&);

    bool is_final() const;

    void set_is_final(bool);
  };
}
```

§ 5.9. `net::message`

This object represents a message sent or received on the network. Messages have an associated context, data, and a flag denoting whether this message can be considered complete (`is_complete`). Messages that are marked as complete will close the underlying connection if the context associated with the message is marked as final. For example, TCP will issue a FIN if a message is marked as complete and the context associated with the message is marked as final.

```
namespace net {

class message {
public:
  message();
  message(buffer, bool is_complete = true);
  message(buffer, context, bool is_complete = true)
  message(const message&);
  message(message&&);
  message& operator=(const message&);
  message& operator=(message&&);

  template<typename Protocol>
  typename Protocol::metadata metadata();
  template<typename Protocol>
  const typename Protocol::metadata metadata() const;

  context get_context() const;
  buffer data() const;
  bool is_complete() const;
};

}
```

§ 5.10. `net::parameters`

This object represents parameters for network protocols.

```
namespace net {

class parameters {
public:
  enum class multipath_service_type {
    disabled,
    handover,
    interactive,
    aggregate
  };
  enum class expired_dns_behavior {
    system_default,
    allow,
    prohibit
  };

  static parameters tls();
  static parameters dtls();

  static parameters tcp();
  static parameters udp();

  parameters(protocol::security::options, protocol::tcp::options);
  parameters(protocol::security::options, protocol::udp::options);

  std::vector<protocol::options> application_protocols() const;
  parameters& set_application_protocols(const std::vector<protocol::opt

  interface require_interface() const;
  parameters& set_require_interface(interface);

  enum interface::type required_interface_type() const;
  parameters& set_required_interface_type(enum interface::type);

  std::vector<interface> prohibited_interfaces() const;
  parameters& set_prohibited_interfaces(const std::vector<interface>&);

  std::vector<enum interface::type> prohibited_interface_types() const;
  parameters& set_prohibited_interface_types(const std::vector<enum int

  bool prohibit_constrained_paths() const;
  parameters& set_prohibit_constrained_paths(bool);
```

```
    bool prefer_no_proxies() const;
    parameters& set_prefer_no_proxies(bool);

    endpoint required_local_endpoint() const;
    parameters& set_required_local_endpoint(endpoint);

    bool allow_local_endpoint_reuse() const;
    parameters& set_allow_local_endpoint_reuse(bool);

    bool accept_local_only() const;
    parameters& set_accept_local_only(bool);

    bool include_peer_to_peer() const;
    parameters& set_include_peer_to_peer(bool);

    enum multipath_service_type multipath_service_type() const;
    parameters& set_multipath_service_type(bool);

    bool allow_fast_open() const;
    parameters& set_allow_fast_open(bool);

    enum expired_dns_behavior expired_dns_behavior() const;
    parameters& set_expired_dns_behavior(enum expired_dns_behavior);
  };

  }
```

§ 5.11. `net::path`

This object represents the known information about a local interface and routes that may be used to send and receive data.

```
namespace net {

class path {
public:
  enum class status {
    satisfied,
    unsatisfied,
    requires_connection,
  };

  path(const path&);
  path(path&&)
  path& operator=(const path&);
  path& operator=(path&&);
  bool operator==(const path&) const;

  std::vector available_interfaces() const;
  bool is_constrained() const;
  bool has_ipv4() const;
  bool has_ipv6() const;
  bool has_dns() const;
  std::vector<endpoint> gateways() const;
  endpoint local_endpoint() const;
  endpoint remote_endpoint() const;
  bool uses_interface_type(enum interface::type type) const;
  status status() const;
};

}
```

§ 5.12. `net::path_monitor`

This object can be used to listen to events related to changing network conditions.

```
namespace net {

class path_monitor {
public:
  path_monitor(workqueue queue);
  path_monitor(enum interface::type type, workqueue queue);

  template<typename Handler, typename =
    std::enable_if_t<std::is_invocable_r_v<void, Handler, path>>>
  path_monitor& on_update(Handler&&);
};

}
```

§ 5.13. `net::protocol`

These objects contain the network protocol definitions, as well as their metadata and options.

```
namespace net {

class protocol {
public:
  class definition {
  public:
    bool operator==(const definition&) const;
  };

  class options {
  };

  class metadata {
  protected:
    metadata(message, definition);
  };

  class ip;
  class tcp;
  class udp;
  class security;
};

class protocol::ip : public protocol {
public:
  static protocol::definition definition();

  class options : public protocol::options {
  public:
    enum class version {
      any,
      v4,
      v6
    };

    version version() const;
    options& set_version(enum version);

    std::uint8_t hop_limit() const;
    options& set_hop_limit(std::uint8_t);

    bool use_minimum_mtu() const;
```

```
      options& set_use_minimum_mtu(bool);

      bool disable_fragmentation() const;
      options& set_disable_fragmentation(bool);

      bool should_calculate_receive_time() const;
      options& set_should_calculate_receive_time(bool);
    };

    enum class ecn {
      non_ect,
      ect0,
      ect1,
      ce
    };

    enum class service_class {
      best_effort,
      background,
      interactive_video,
      interactive_voice,
      responsive_data,
      signaling
    };

    class metadata : public protocol::metadata {
    public:
      metadata();
      metadata(message);
      metadata(const metadata&);
      metadata(metadata&&);
      metadata& operator=(const metadata&);
      metadata& operator=(metadata&&);

      ecn ecn() const
      metadata& set_ecn(enum ecn);

      service_class service_class() const;
      metadata& set_service_class(enum service_class);

      std::uint64_t receive_time() const;
    };
  };
```

```
class protocol::tcp : public protocol {
public:
  static protocol::definition definition();

  class options : public protocol::options {
  public:
    bool no_delay() const;
    options& set_no_delay(bool);

    bool no_push() const;
    options& set_no_push(bool);

    bool no_options() const;
    options& set_no_options(bool);

    bool keep_alive() const;
    options& set_keep_alive(bool);

    std::size_t keep_alive_count() const;
    options& set_keep_alive_count(std::size_t);

    std::chrono::seconds keep_alive_idle() const;
    options& set_keep_alive_idle(std::uint32_t);

    std::chrono::seconds keep_alive_interval() const;
    options& set_keep_alive_interval(std::chrono::seconds);

    std::size_t maximum_segment_size() const;
    options& set_maximum_segment_size(std::size_t);

    std::chrono::seconds connection_timeout() const;
    options& set_connection_timeout(std::chrono::seconds);

    std::chrono::seconds persist_timeout() const;
    options& set_persist_timeout(std::chrono::seconds);

    std::chrono::seconds connection_drop_time() const;
    options& set_connection_drop_time(std::chrono::seconds);

    bool retransmit_fin_drop() const;
    options& set_retransmit_fin_drop(bool);
```

```
    bool disable_ack_stretching() const;
    options& set_disable_ack_stretching(bool);

    bool enable_fast_open() const;
    options& set_enable_fast_open(bool);

    bool disable_ecn() const;
    options& set_disable_ecn(bool);
  };

  class metadata : protocol::metadata {
  public:
    std::uint32_t available_receive_buffer() const;
    std::uint32_t available_send_buffer() const;
  };
};

class protocol::udp : public protocol {
public:
  static protocol::definition definition();

  class options : public protocol::options {
  public:
    bool prefer_no_checksum() const;
    options& set_prefer_no_checksum(bool);
  };

  class metadata : public protocol::metadata {
  };
};

class protocol::security : public protocol {
public:
  static definition definition();

  class certificate {
  public:
    certificate();
    certificate(std::string_view);
    certificate(const certificate&);
    certificate(certificate&&);
    certificate& operator=(const certificate&);
    certificate& operator=(certificate&&);
```

```
    std::string common_name() const;
    std::vector<std::uint8_t> der_bytes() const;
    std::vector<std::uint8_t> public_key() const;
  };

  class identity {
  public:
    identity();
    identity(const std::vector<certificate>&, std::string_view);
  };

  class options : public protocol::options {
  public:
    identity get_local_identity() const;
    options& set_local_identity(identity);

    template<typename Handler, typename =
      std::enable_if_t<std::is_invocable_r_v<void, Handler,
        metadata, std::function<void(bool)>>>>
    options& on_verify(Handler, workqueue);
  };

  class metadata : public protocol::metadata {
  public:
    std::string_view negotiated_protocol() const;
    bool early_data_accepted() const;
    std::vector<certificate> certificate_chain() const;
    std::string_view server_name() const;

    template<typename Handler, typename =
      std::enable_if_t<std::is_invocable_r_v<void, Handler, buffer>>>
    bool oscp_response(Handler) const;
  };
};

}

#endif
```

§ 5.14. `net::protocol::framer`

A framer defines a protocol in a connection's protocol stack that parses and writes messages on top of a transport protocol, such as a TCP stream. A framer can add and parse headers or delimiters around application data to provide a message-oriented abstraction.

The framer is templated with an implementation class that must have the following traits:

- `class T::options`

  An options class derived from `protocol::framer<T>` that defines protocol specific options

- `class T::metadata`

  A metadata class derived from `protocol::framer<T>` that defines protocol specific metadata

- `start_result T::start()`

  This will be called during connection establishment. Return `start_result::ready` to indicate that the framer instance is ready to begin sending/receiving data. Return `start_result::will_mark_ready` to indicate that the framer instance needs to complete handshaking before it is able to send/receive data. If there is no `T::start` it is assumed no handshake is necessary, which will behave the same as if `T::start` always returned `start_result::ready`.

- `size_t T::handle_input()`

  This function will be invoked whenever new input data is available to be parsed. When this block is run, the implementation should call functions like `parse_input()` and `deliver_input()`.

  Each invocation represents new data being available to read from the network. This data may be insufficient to complete a message, or may contain multiple messages. Implementations are expected to try to parse messages in a loop until parsing fails to read enough to continue.

  Return a hint of the number of bytes that should be present before invoking this handler again. Returning 0 indicates that the handler should be invoked once any data is available.

- `void T::handle_output(net::protocol::metadata, size_t message_length, bool is_complete)`

  This function will be invoked whenever an output message is ready to be sent. When this block is run, the implementation should call functions like `parse_output()` and `write_output()`.

Each invocation represents a single complete or partial message that is being sent. The implementation is expected to write this message or let it be dropped in this handler.

- `bool T::stop()`

  This function will be invoked when the connection is being disconnected, to allow the framer implementation a chance to send any final data.

  Return true if the framer is done and the connection can be fully disconnected, or false the stop should be delayed. If false, the implementation must later call `mark_failed(std::error_code)` on the instance.

- `void T::wakeup()`

  This optional function will be invoked whenever the wakeup timer set via `schedule_wakeup()` fires. This is intended to be used for sending keepalives or other control traffic.

```
namespace net {

template<typename T> class protocol::framer
public:
  enum class start_result {
    ready,
    will_mark_ready
  };

  static protocol::definition definition();
  class options : public protocol::options {
  public:
    options();
  };

  class metadata : public protocol::metadata {
  public:
    metadata();
    metadata(framer);

    template<typename Value>
    std::optional<Value> get_value(std::string_view key) const;

    template<typename Value>
    void set_value(std::string_view key, Value);
  };

  framer();

  void mark_ready();
  void mark_failed(std::error_code);

  template<typename Parser, typename =
    std::enable_if_t<std::is_invocable_r_v<void, Parser,
      const void*, std::size_t, bool>>>
  bool parse_input(std::size_t min_length, std::size_t max_length, Pars

  void deliver_input(const void*, size_t, metadata, bool is_complete);
  bool deliver_input_no_copy(size_t, metadata, bool is_complete);
  void pass_through_input();

  template<typename Parser, typename =
```

```
      std::enable_if_t<std::is_invocable_r_v<void, Parser,
        const void*, std::size_t, bool>>>
    bool parse_output(std::size_t min_length, std::size_t max_length, Par

    void write_output(const void*, std::size_t);
    void write_output_no_copy(size_t);
    void pass_through_output();
    void schedule_wakeup(std::chrono::milliseconds);

    template<typename Handler, typename =
      std::enable_if_t<std::is_invocable_r_v<void, Handler>>>
    void async(Handler&&);

    endpoint remote_endpoint() const;
    endpoint local_endpoint() const;
    parameters parameters();
    void prepend_appplication_protocol(protocol::options);
  };

  }
```

§ 5.15. `net::workqueue`

This object is basically an executor that works well on Apple platforms. We should just use executor instead, but here's what this suggested API was developed with. This is not the important part of this suggested API.

```
namespace net {

class workqueue {
public:
  static void main();
  static workqueue main_queue();

  workqueue(const workqueue&);
  workqueue(workqueue&&);
  workqueue& operator=(const workqueue&);
  workqueue& operator=(workqueue&&);

  template<typename Work, typename =
    std::enable_if_t<std::is_invocable_r_v<void, Work>>>
  void dispatch(Work)
};

}
```

6. Examples

6.1. Client

A simple TCP/HTTP client can be written using coroutines. This example uses `std::lazy` from [p1056r1].

```cpp
#include <iostream>
#include <net>

std::lazy<void> run()
{
  net::workqueue queue(net::workqueue::main_queue());
  net::endpoint::host host("www.apple.com", 80);
  net::connection connection(host, net::parameters::tcp(), queue);
  connection.start();

  std::cout << "Sending request" << std::endl;
  net::message message(net::buffer("GET / HTTP/1.1\r\nHost: www.apple.c
  auto sendResult = co_await connection.send(message);
  if (!sendResult) {
    std::cerr << "failed to send request" << std::endl;
    co_return;
  }

  std::cout << "Sent request, waiting for response" << std::endl;
  auto message = co_await connection.receive();
  if (!message) {
    std::cerr << "failed to receive response" << std::endl;
    co_return;
  }

  std::cout << "Received response" << std::endl;
  message->data().get([](const uint8_t *bytes, std::size_t size) {
    std::cout << std::string(reinterpret_cast<const char *>(bytes), siz
  });
  std::cout << std::endl;
  co_return;
}

int main(int, char**)
{
  auto lazy = run();
  net::workqueue::main();
}
```

All that is needed to make the request use TLS is to change `net::parameters::tcp()` to `net::parameters::tls()`. Port 80 should also be changed to port 443 for this particular server.

§ 6.2. Server

Using this suggested API, one can make a simple TCP/HTTP server as follows:

```cpp
#include <iostream>
#include <net>

int main(int, char**)
{
  net::parameters listener_parameters = net::parameters::tcp();

  net::listener listener(listener_parameters, net::workqueue::main_queu
  listener.on_new_connection([] (auto connection) {
    std::cout << "Received new connection from a client!" << std::endl;
    connection.start();

    connection.receive().then([connection] (auto request) mutable {
      if (!request) {
        std::cerr << "Failed to receive request" << std::endl;
        return;
      }

      std::cout << "Received request:" << std::endl;
      request->data().get([](const uint8_t *bytes, std::size_t size) {
        std::cout << std::string(reinterpret_cast<const char *>(bytes),
      });
      std::cout << std::endl;

      std::cout << "Sending response" << std::endl;
      net::buffer buffer(
        "HTTP/1.1 200 OK\r\nContent-Length: 23\r\n\r\n"
        "The server says hello!\n"
      );
      connection.send(net::message(buffer)).then([] (auto result) {
        if (!result) {
          std::cerr << "Failed to send response" << std::endl;
          return;
        }
        std::cout << "Successfully sent response" << std::endl;
      });
    });
  });

  listener.on_state_changed([listener](auto state, auto error) {
    if (!error && state == net::listener::state::ready) {
      std::cout << "server is now ready and listening. ";
```

```
        std::cout << "curl http://127.0.0.1:" << listener.port();
        std::cout << "/ -v" << std::endl;
      }
    });

    listener.start();

    net::workqueue::main();
  }
```

Which can be changed into a TLS server by adding these lines after the declaration of `listener_parameters`:

```
// This is a test RSA private key from BoringSSL.
auto rsa_private_key = base64_decode(
  "MIICXgIBAAKBgQDYK8imMuRi/03z0K1Zi0WnvfFHvwlYeyK9Na6XJYaUoIDAtB92"
  "kWdGMdAQhLciHnAjkXLI6W15OoV3gA/ElRZ1xUpxTMhjP6PyY5wqT5r6y8FxbiiF"
  "KKAnHmUcrgfVW28tQ+0rkLGMryRtrukXOgXBv7gcrmU7G1jC2a7WqmeI8QIDAQAB"
  "AoGBAIBy09Fd4DOq/Ijp8HeKuCMKTHqTW1xGHshLQ6jwVV2vWZIn9aIgmDsvkjCe"
  "i6ssZvnbjVcwzSoByhjN8ZCf/i15HECWDFFh6gt0P5z0MnChwzZmvatV/FXCT0j+"
  "WmGNB/gkehKjGXLLcjTb6dRYVJSCZhVuOLLcbWIV10gggJQBAkEA8S8sGe4ezyyZ"
  "m4e9r95g6s43kPqtj5rewTsUxt+2n4eVodD+ZUlCULWVNAFLkYRTBCASlSrm9Xhj"
  "QpmWAHJUkQJBAOVzQdFUaewLtdOJoPCtpYoY1zd22eae8TQEmpGOR11L6kbxLQsk"
  "aMly/DOnOaa82tqAGTdqDEZgSNmCeKKknmECQAvpnY8GUOVAubGR6c+W90iBuQLj"
  "LtFp/9ihd2w/PoDwrHZaoUYVcT4VSfJQog/k7kjE4MYXYWL8eEKg3WTWQNECQQDk"
  "104Wi91Umd1PzF0ijd2jXOERJU1wEKe6XLkYYNHWQAe5l4J4MWj9OdxFXAxIuuR/"
  "tfDwbqkta4xcux67//khAkEAvvRXLHTaa6VFzTaiiO8SaFsHV3lQyXOtMrBpB5jd"
  "moZWgjHvB2W9Ckn7sDqsPB+U2tyX0joDdQEyuiMECDY8oQ=="
);

// This is a test self-signed certificate from BoringSSL.
auto certificate_bytes = base64_decode(
  "MIICWDCCAcGgAwIBAgIJAPuwTC6rEJsMMA0GCSqGSIb3DQEBBQUAMEUxCzAJBgNV"
  "BAYTAkFVMRMwEQYDVQQIDApTb21lLVN0YXRlMSEwHwYDVQQKDBhJbnRlcm5ldCBX"
  "aWRnaXRzIFB0eSBMdGQwHhcNMTQwNDIzMjA1MDQwWhcNMTcwNDIyMjA1MDQwWjBF"
  "MQswCQYDVQQGEwJBVTETMBEGA1UECAwKU29tZS1TdGF0ZTEhMB8GA1UECgwYSW50"
  "ZXJuZXQgV2lkZ2l0cyBQdHkgTHRkMIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKB"
  "gQDYK8imMuRi/03z0K1Zi0WnvfFHvwlYeyK9Na6XJYaUoIDAtB92kWdGMdAQhLci"
  "HnAjkXLI6W15OoV3gA/ElRZ1xUpxTMhjP6PyY5wqT5r6y8FxbiiFKKAnHmUcrgfV"
  "W28tQ+0rkLGMryRtrukXOgXBv7gcrmU7G1jC2a7WqmeI8QIDAQABo1AwTjAdBgNV"
  "HQ4EFgQUi3XVrMsIvg4fZbf6Vr5sp3Xaha8wHwYDVR0jBBgwFoAUi3XVrMsIvg4f"
  "Zbf6Vr5sp3Xaha8wDAYDVR0TBAUwAwEB/zANBgkqhkiG9w0BAQUFAAOBgQA76Hht"
  "ldY9avcTGSwbwoiuIqv0jTL1fHFnzy3RHMLDh+Lpvolc5DSrSJHCP5WuK0eeJXhr"
  "T5oQpHL9z/cCDLAKCKRa4uV0fhEdOWBqyR9p8y5jJtye72t6CuFUV5iqcpF4BH4f"
  "j2VNHwsSrJwkD4QUGlUtH7vwnQmyCFxZMmWAJg==="
);

net::protocol::security::certificate certificate(certificate_bytes);
net::protocol::security::identity identity({ certificate }, rsa_private_
net::protocol::security::options tls_options;
tls_options.set_local_identity(identity);
listener_parameters.set_application_protocols({ tls_options });
```

`http` would need to be changed to `https` and `-v` would need to be changed to `-v --insecure` to allow curl to connect to this server successfully. It requires curl's `insecure` flag because it uses a self-signed certificate instead of a certificate chain with a trusted root certificate authority. This is useful for local development, but obviously requires a certificate for production usage.

§ 6.3. Path Listener

This is one of the great advantages of this suggested API. It allows people to write programs that are common today but unheard of in the 1980's when sockets were designed.

```
#include <iostream>
#include <net>

std::ostream& operator<<(std::ostream& os, const net::path& path)
{
  for (auto& interface : path.available_interfaces())
    os << "interface: " << interface.name() << std::endl;
  return os;
}

int main(int, char **)
{
  net::path_monitor monitor(net::workqueue::main_queue());
  monitor.on_update([](auto path) {
    std::cout << "path update: " << std::endl << path;
  });

  net::workqueue::main();
}
```

This simple program will print status updates when the network interfaces changes, such as when WiFi is connected or when a mobile device enters an area without cellular data availability.

§ 6.4. Protocol Framer

This is an implementation of a subset of HTTP. It allows setting method, path, and header fields of HTTP requests, and it parses the response header all as one string in the metadata and the passes the

body through to `connection.receive()`. This example also uses `std::lazy` from [p1056r1].

```
#include <net>
#include <stdio.h>
#include <unordered_map>
#include <sstream>

class http {
public:
  using framer = net::protocol::framer<http>;

  enum class method { get, post };

  enum class response_parsing_state {
    looking_for_first_carriage_return,
    looking_for_first_newline,
    looking_for_second_carriage_return,
    looking_for_second_newline,
    body
  };

  static std::string_view label() { return "http"; }
  static net::protocol::definition definition() { return framer::defini
  class options : public framer::options { };

  class metadata : public framer::metadata {
  public:
    using framer::metadata::metadata;

    std::string path() const
    {
      if (auto value = get_value<std::string>("path"))
        return std::move(*value);
      return "/";
    }
    metadata& set_path(std::string path)
    {
      set_value("path", path);
      return *this;
    }

    method method() const
    {
      if (auto value = get_value<enum method>("method"))
```

```
        return std::move(*value);
      return method::get;
    }
    metadata& set_method(enum method method)
    {
      set_value("method", method);
      return *this;
    }

    using header_fields_t = std::unordered_map<std::string, std::string>
    header_fields_t header_fields() const
    {
      if (auto value = get_value<header_fields_t>("header_fields"))
        return std::move(*value);
      return { };
    }
    metadata& set_header_fields(header_fields_t fields)
    {
      set_value("header_fields", fields);
      return *this;
    }

    std::string response_header() const
    {
      if (auto value = get_value<std::string>("response_header"))
        return std::move(*value);
      return { };
    }
    metadata& set_response_header(std::string header)
    {
      set_value("response_header", header);
      return *this;
    }
  };

  http(framer framer) : m_framer(std::move(framer)) { }

  size_t handle_input();

  void handle_output(net::protocol::metadata, size_t, bool);

private:
  framer m_framer;
```

```
    response_parsing_state m_state {
      response_parsing_state::looking_for_first_carriage_return
    };

    metadata m_response_metadata;
    std::ostringstream m_response_header_stream;
  };

  size_t http::handle_input()
  {
    auto ok = m_framer.parse_input(1, std::numeric_limits<size_t>::max(),
      [&](const void *buffer, size_t length, bool is_complete) -> size_t
      // "\r\n\r\n" indicates the end of the header and beginning of body
      // which we want to deliver. This is just a state machine looking fo
      // which may be in two different input pieces.
      const char* char_buffer = static_cast<const char*>(buffer);
      for (size_t i = 0; i < length; ++i) {
        char c = char_buffer[i];
        m_response_header_stream << c;
        switch (m_state) {
          case response_parsing_state::looking_for_first_carriage_return:
            if (c == '\r')
              m_state = response_parsing_state::looking_for_first_newline
            break;
          case response_parsing_state::looking_for_first_newline:
            if (c == '\n')
              m_state = response_parsing_state::looking_for_second_carriag
            else
              m_state = response_parsing_state::looking_for_first_carriage
            break;
          case response_parsing_state::looking_for_second_carriage_return
            if (c == '\r')
              m_state = response_parsing_state::looking_for_second_newline
            else
              m_state = response_parsing_state::looking_for_first_carriage
            break;
          case response_parsing_state::looking_for_second_newline:
            if (c == '\n') {
              m_state = response_parsing_state::body;
              m_response_metadata.set_response_header(m_response_header_st
            } else
              m_state = response_parsing_state::looking_for_first_carriage
            break;
```

```
          case response_parsing_state::body:
            m_framer.deliver_input(
              char_buffer + i,
              length - i,
              m_response_metadata,
              is_complete
            );
            return length;
        }
      }
      return length;
    });

    if (!ok)
      m_framer.mark_failed(make_error_code(std::errc::protocol_error));

    return 0;
  }

  void handle_output(
    net::protocol::metadata metadata,
    size_t message_length,
    bool is_complete)
  {
    auto method_string = [] (auto method) {
      switch (method) {
      case method::get:
        return "GET";
      case method::post:
        return "POST";
      }
    };

    auto http_metadata = dynamic_cast<const http::metadata&>(metadata);
    std::ostringstream os;
    os << method_string(http_metadata.method())
      << " " << http_metadata.path() << " HTTP/1.1\r\n";

    auto header_fields = http_metadata.header_fields();
    for (auto& kv : header_fields)
        os << kv.first << ": " << kv.second << "\r\n";
    os << "\r\n";
```

```
    auto header = os.str();
    m_framer.write_output(
      reinterpret_cast<const uint8_t *>(header.c_str()), header.size());
  }

  std::lazy<void> run()
  {
    net::parameters parameters(net::parameters::tcp());
    parameters.set_application_protocols({
      net::protocol::security::options(),
      http::options()
    });
    net::connection connection(
      net::endpoint::host("www.apple.com", 443),
      parameters,
      net::workqueue::main_queue()
    );
    connection.start();
    std::cout << "Sending request" << std::endl;
    http::metadata http_metadata;
    http_metadata.set_method(http::method::get).set_path("/").set_header_
      { "Host", "www.apple.com" },
      { "Connection", "Close" }
    });
    auto sendResult = co_await connection.send({ http_metadata });
    if (!sendResult) {
      std::cerr << "failed to send request" << std::endl;
      co_return;
    }
    std::cout << "Sent request, waiting for response" << std::endl;
    auto message = co_await connection.receive();
    if (!message) {
      std::cerr << "failed to receive response" << std::endl;
      co_return;
    }
    auto received_metadata = message->metadata<http>();
    std::cout << "http response header:" << std::endl;
    std::cout << received_metadata.response_header() << std::endl;
    std::cout << "http response body:" << std::endl;
    message->data().get([](const uint8_t *bytes, std::size_t size) {
      std::cout <<
        std::string(reinterpret_cast<const char *>(bytes), size)
        << std::endl;
```

```
    });
    co_return;
  }

  int main(int, char**)
  {
    auto lazy = run();
    net::workqueue::main();
    return 0;
  }
```

§ 7. Notes

Identities consisting of certificate chains and private keys are commonly used, but there are also things like hardware keys which sign data upon request but never allow access to the private key. This will need to be developed further to support such devices.

TLS version and cipher suite are intentionally not exposed because they would not allow crypto agility without breaking ABI. Users who need such control will have to implement their own TLS on top of TCP or DTLS on top of UDP.

Relatedly, the allowance for custom transport and application protocols has not yet been included but can. It is tricky to do so without mandating interitance from STL types, which would be undesirable.

The storage mechanism on `framer::metadata` is not optimal because it requires a string key to set or access its values. Don't let that be a distraction from the design of `framer` allowing implementation of custom protocols.

§ 8. Implementability

We implemented the suggested API using Apple's Network.framework API, available on iOS and macOS. This implementation currently stands at 2350 lines of code (without counting `std::expected`). All examples function as described.

Undue implementation burden was a severe concern in the Committee, particularly when it comes to the cryptography used in modern secure networking protocols. Our implementation needs little to no experience with cryptography. Indeed, the implementation simply needs to wrap existing APIs. Our

experience makes us confident that modern operating systems and userspace libraries are more than adequate to implement this suggested API fully.

Much of this can be implemented on top of the Berkeley Sockets API. It's merely a different shape for the API with the important addition of metadata, which provides a mechanism to configure protocol properties in a way that can be elegantly extended to protocols with security in the transport layer, such as QUIC.

The `path_monitor` class requires OS support that many operating systems already have internally.

§ References

§ Informative References

[N4771]
Jonathan Wakely. Working Draft, C++ Extensions for Networking. 8 October 2018. URL: https://wg21.link/n4771

[P0323R7]
Vicente Botet, JF Bastien. std::expected. 22 June 2018. URL: https://wg21.link/p0323r7

[P1056R1]
Lewis Baker, Gor Nishanov. Add lazy coroutine (coroutine task) type. 7 October 2018. URL: https://wg21.link/p1056r1

[p1860R0]
JF Bastien, Alex Christensen. C++ Networking Must Be Secure By Default. 7 October 2019. URL: https://wg21.link/p1860r0

[P1861R0]
JF Bastien, Alex Christensen. Secure Connections in Networking TS. 7 October 2019. URL: https://wg21.link/p1861r0